

目次

気象データ×生成 AI — バイブコーディング実践記

上原政博（沖縄気象台／WXBC 個人会員）

2026 年 9 月 18 日 WXBC 人材育成 WG 全体会合 話題提供 原稿案

はじめに

プログラミングを少しかじった程度で、開発の専門知識はもっていません。そんな私ですが、WXBC の「アメダス気象データ分析チャレンジ」で Python と機械学習を初めて学び、ちょうど同じ時期に始めた生成 AI によるプログラミングと組み合わせることで、新しい世界が広がりました。

AI と対話しながらアプリ開発を進める「バイブコーディング（Vibe Coding）」というスタイルで、（手抜きではありますが）プログラミングの楽しさを知りました。今年 3 月に生成 AI エージェントの Claude Code を使い始めてからは、面倒な「作業」を減らし、アイデアを形にする高速開発のサイクルが定着しています。

本稿では、この開発スタイルの紹介と、農研機構メッシュ農業気象データを活用した太陽光発電量予測の機械学習事例を報告します。取り上げる内容は次の 5 つです。

1. 生成 AI 活用の経緯
 2. 開発したプロダクト（気象・予測ツール）
 3. 農研機構メッシュ農業気象データ × AI による機械学習の取り組み
 4. バイブコーディングとは
 5. まとめ・今後の展開
-

1. 生成 AI 活用の経緯 — チャットから Claude Code へ

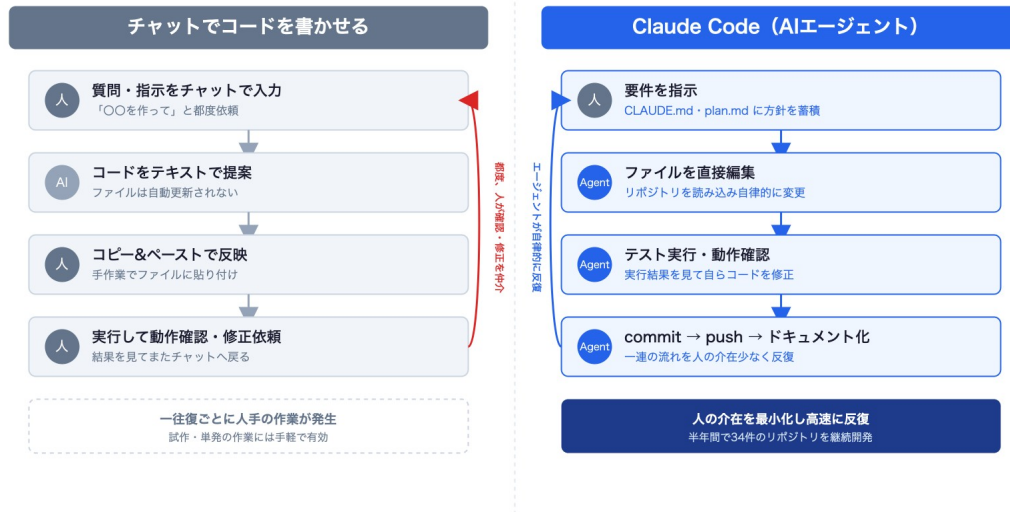
生成 AI を使ったコード作成を始めたのは約 2 年前です。ChatGPT を相手に、チャットベースで Web アプリの試作を繰り返していました。その後、WXBC の「アメダス気象データ分析チャレンジ」を受講し、Python と機械学習、気象データの扱い方を学びました。さらに AI エージェントという使い方があることを知り、今年 3 月から Claude Code での開発に切り替えています。

合言葉は「何でも自動化」です。たとえば、電気料金を電力会社の Web サイトで毎日確認する手間を自動化しました。データの自動取得・グラフ化・見える化を行い、結果を Web に載せています。開発の進め方は、AI へのチャット指示でコードを自動修正し、ドキュメント化して GitHub へ push する、という一連の流れをひたすら反復して改良していくものです。天気図、発電量予測、電力データ分析、ローカル LLM の比較など、実用ニーズに応じてテーマを設定してきました。

チャットでの開発と Claude Code (AI エージェント) の違い

チャットによる開発では、継続的・安定的に開発を進めるのが難しいと感じていました。毎回状況を説明し、プロンプトで指示を出し、コードを貼り付けてから作業を再開しなければなりません。一往復ごとに人手の作業が発生します。

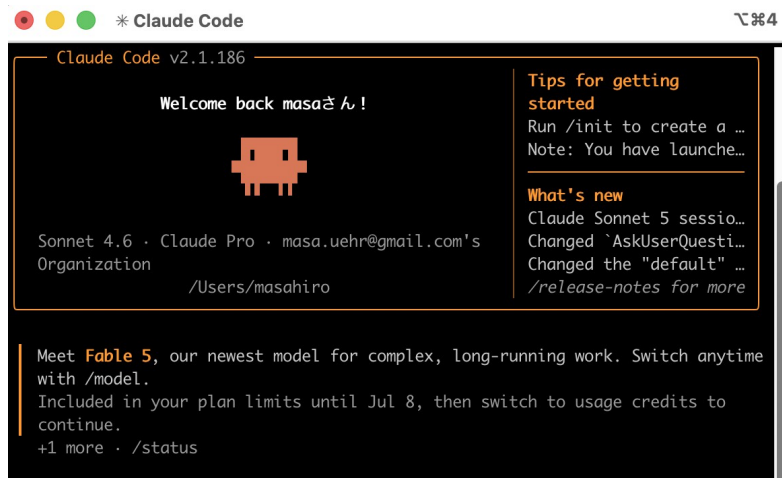
Claude Code では、プロンプトに相当する Markdown ファイルを常に読み込ませた状態で起動します。コードは git で管理し、コードとドキュメントを GitHub に保存します。これにより、スマホからドキュメントを確認したり、毎日の自動処理が出力したグラフを見たりできるようになりました。



チャットでコードを書かせる方式と、Claude Code (AI エージェント) 方式の比較。チャット方式は一往復ごとに人手の作業（コピー&ペースト、動作確認、修正依頼）が挟まる。エージェント方式は要件の指示以降、ファイル編集・テスト実行・commit/pushまでを人の介入を最小化して反復する。

Claude Code の具体的な使い方

- 作業フォルダに Markdown ファイル（CLAUDE.md、plan.md など）を保存し、制限事項（セキュリティ）や開発方針を AI に読ませてから開発を始めます。
- チャットで指示を出すと、AI が直接 Python コードを書いてファイルに保存します。タイプミスありません。
- チャットのやり取り（バグ取り、機能追加、Q&A）は常にまとめさせてマニュアル化します。チャットだと自分でやるのが面倒な作業ですが、AI 頼みなら簡単です。
- AI が作成した Python コードとドキュメントを GitHub へ push する。この流れを反復して改良していきます。



Claude Code の起動画面

2. サンプルコードを土台にした開発（応用・高機能化）

WXBC「気象データ分析チャレンジ」で、Python・気象データの取り扱い・機械学習の基礎を習得しました。この講座を通じて、**農研機構メッシュ農業気象データシステム（AMGSDS）**の存在も知りました。

天気図まわりでは、黒良氏の note 連載記事を参考に、GRIB2 データの読み込み処理と天気図作成スクリプトを実装しました。Web アプリについては、気象台職員が書いた JavaScript のコードを AI に読ませて Web アプリを作成しています。Python と JavaScript の相互変換も簡単で、公開されている教材コードをベースに Claude Code で機能追加・改修を進めました。

つまり、教材やサンプルコードを土台として、Claude Code と一緒に機能を拡張していく開発スタイルです。

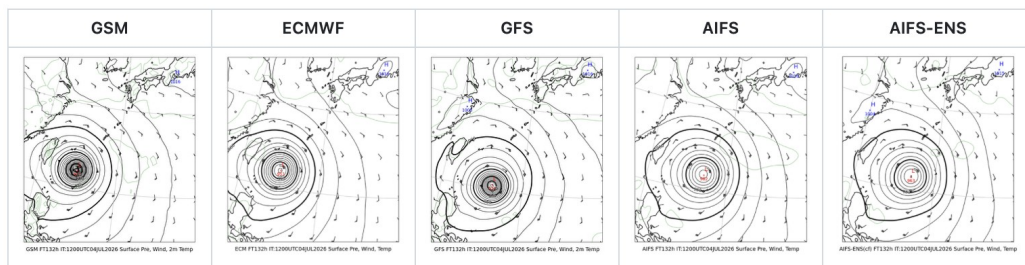
3. 開発したプロダクト（気象・予測ツール）

5 気象モデルによる台風進路予測の比較

GSM（気象庁）、ECMWF（欧州中期予報センター）、GFS（米国）、AIFS、AIFS-ENS（ECMWF の AI 予測）による地上気圧予報を並列表示するツールです。下図は FT=132h 予報（7 月 10 日 9 時 JST 時点）の例です。

GSM を含む 5 モデルの比較表示は独自開発で、台風接近時の進路確認に活用しています。カラー版天気図、極天気図、5 日平均、エマグラム、断面図など、高度な天気図作成も可能で、スクリプトは GitHub で公開しています。

FT=132h (7/10 9時 JST)



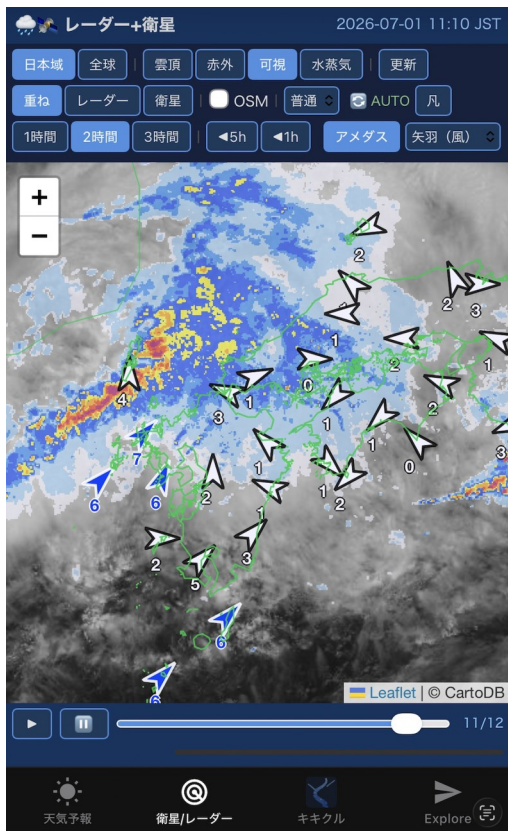
5 モデルによる地上気圧予報の並列表示（FT=132h、7/10 9 時 JST 時点）

衛星画像・レーダー表示アプリ、潮位観測表示アプリ

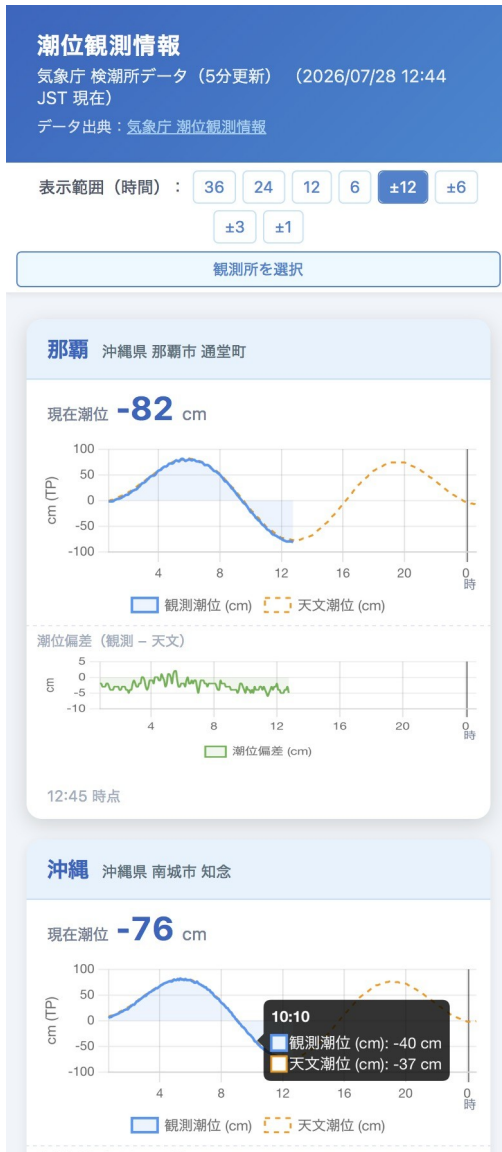
気象情報表示 Web アプリのコードを Claude Code に読み込ませ、機能単位に分割したサンプルアプリ集を作りました。単機能のコードを AI に読ませて簡単なアプリを作り、そこから AI で機能を追加したりバグを取ったりします。

Web アプリ・iOS アプリ・Android アプリを同時に実装しました（Expo Go を利用。現時点では Web アプリのみ公開）。既存の実装を参考に、AI で機能を移植・改修する開発手法です。気象庁の検潮所データをリアルタイム表示する潮位観測アプリも、同じ手法で開発しました。

- レーダー・衛星画像アプリ: <https://masauehr.github.io/iphone-weather-app/radar>
- 潮位観測アプリ: https://masauehr.github.io/tide_viewer/



レーダー・衛星画像表示アプリ (Web 版)



潮位観測表示アプリ (気象庁の検潮所データをリアルタイム表示)

気象庁データ取得 MCP サーバー (jma-mcp)

気象庁サイト (bosai.jma.go.jp) が内部で使用している非公開 API (認証不要) を AI と一緒に解析し、MCP サーバーとして実装しました。天気予報・警報/注意報・早期注意情報・観測ランキング・潮位など、計 21 種類のツールを提供しています。HTTP/SSE 版 (jma-mcp-remote) を Render にデプロイしており、Claude.ai の Web 版・デスクトップアプリ・iPhone 版からも利用できます。

- GitHub: <https://github.com/masauehr/jma-mcp>

Claude Code へ自然言語で質問するだけで、自動的にツールが呼び出され、最新データが取得されます。

「東京の週間天気予報は？」 「沖縄への早期注意情報を教えて」 「那覇の現在の潮位を教えて」 「今日の全国降水量ランキングを見せて」

気象庁 HP からデータを取得するコードは、その後に別のアプリ（検潮所観測データの一覧表示など）を作るときに AI が参考にできるので、開発の高速化につながります。

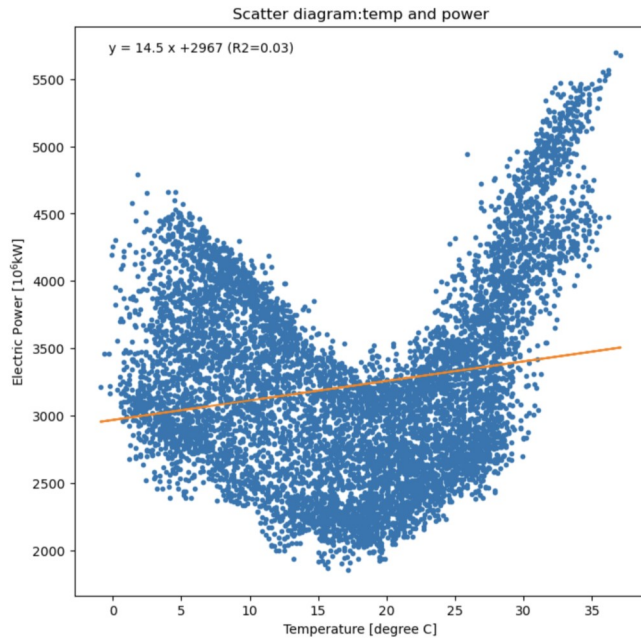
MCP (Model Context Protocol) とは: Anthropic が策定したオープン標準プロトコル。AI (Claude) と外部ツール・データソースを接続する仕組みで、これにより AI が学習データにない最新情報やローカル環境の情報をリアルタイムに取得できるようになる。

4. 農研機構メッシュ農業気象データ × AI による機械学習の取り組み

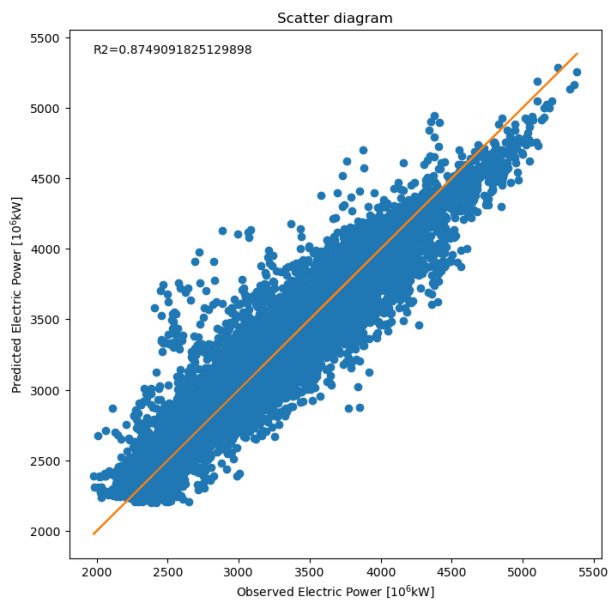
取り組みのきっかけ

WXBC「アメダス気象データ分析チャレンジ」で、気象データ（気温など）から東京電力の電力消費量をニューラルネットワークで予測する演習に取り組みました。気温だけの単回帰では $R^2=0.03$ とほぼ無相関でしたが、複数の気象・時刻要素を使ったニューラルネットワークでは $R^2=0.87$ まで精度が向上しました。

この「気象データ（説明変数）→ 電力消費量（目的変数）」という構造を、自宅の太陽光発電にも応用できないかと考えました。チャレンジで学んだ手法をベースに、農研機構メッシュ農業気象データを使った独自モデルの開発に着手します。これが `ml_forecast` の出発点です。



気温と電力消費量の単回帰 ($R^2=0.03$ 、ほぼ無相関)。WXBC「アメダス気象データ分析チャレンジ」教材より



複数の気象・時刻要素を使ったニューラルネットワークによる予測値と実測値の散布図 ($R^2=0.87$)。WXBC「アメダス気象データ分析チャレンジ」教材より

教師データ（太陽光発電量）の取得を AI で解決

太陽光発電アプリの月次棒グラフのスクリーンショットを画像処理で自動解析し、教師データとなる日別発電量を抽出しています（`extract_hatuden_daily.py`）。

- 緑色のバー（発電量）をピクセル色で検出し、バーの高さから数値化します。
- アプリが表示する月次合計値でスケーリングし、目標値との誤差 ± 0.03 kWh 以内に補正します。
- 画像から発電量を読み取る仕組みと、手入力で発電量を追加するアプリ（Jupyter Notebook）を作り、日々の精度検証にも使っています。

スクリーンショットを撮るだけで教師データが増えます。手入力なしで学習データを日々蓄積できるのが利点です。



太陽光発電アプリの月次棒グラフ（教師データ抽出元）

農業気象データによる太陽光発電量予測

農研機構メッシュ農業気象データシステム（AMGSDS）は、全国をメッシュ単位で網羅する気象データです。WXBC「アメダス気象データ分析チャレンジ」の公開コードを参考にモデルを構築しました。説明変数は全天日射量・日照時間などの気象予報値、目的変数は実測発電量です。毎朝 6:30 に予測を自動更新し、結果は GitHub で確認しています。

モデル	R ² (OOF)	MAE (kWh/日)
Random Forest	0.846	1.89

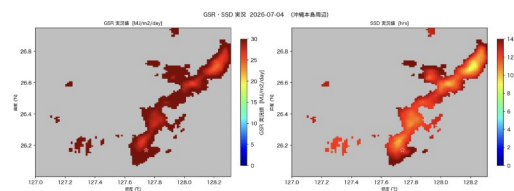
モデル	R ² (OOF)	MAE (kWh/日)
Gradient Boosting	0.840	1.93
Ridge 回帰	0.813	2.14

複数モデルを比較検討し、精度が最も高い Random Forest を現行モデルに採用しました。

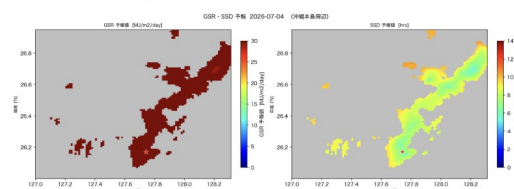
周辺地域の面的分布 (GSR・SSD)

最終更新: 2026-07-05

前日実況 2026-07-04

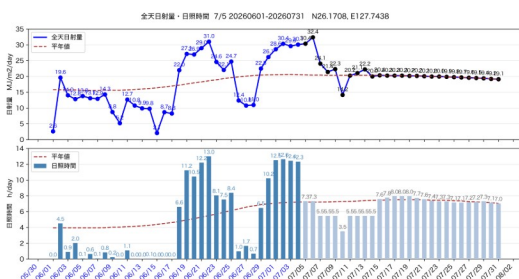


前日の当日予報 2026-07-04 (2026-07-04 生成)



左=GSR・右=SSD ★=自宅地点 (N26.1708, E127.7418) 単位: MJ/m²/day (GSR), h/day (SSD)

農研機構メッシュ農業気象データの面分布



全天日射量 (GSR) の分布例 (2026-07-05)

ml_forecast — 自宅太陽光発電量 機械学習予測

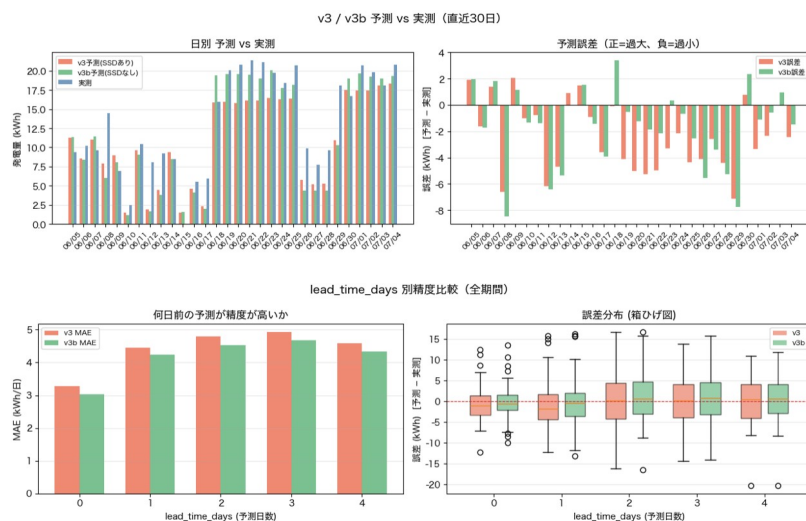
AMGSDS の GSR・GSR/平年比・SSD を説明変数に、自宅の日別太陽光発電量 (kWh) を Random Forest で予測しています。

直近5日間の発電量予測 (最終更新: 2026-07-05)

日付	GSR	平年比	SSD	天気	v3	v3b
07/05(日)	30.4	1.48	7.3	晴れ	18.4	19.3
07/06(月)	32.4	1.59	7.3	晴れ	18.5	20.1
07/07(火)	24.1	1.18	5.5	晴れ	16.8	18.2
07/08(水)	21.4	1.05	5.5	晴れ	16.4	17.2
07/09(木)	22.3	1.10	5.5	晴れ	16.8	18.1

単位: GSR (MJ/m²) ・ SSD (h) ・ v3/v3b (kWh) モデル: Random Forest v3 (CV MAE=2.09 kWh/日)

精度評価は継続中で、日次での予測・検証サイクルを運用しています。



太陽光発電量予測の検証（予測と実測）

運用を通じた説明変数の見直し

初期モデルは、説明変数の選定を含めて生成 AI に一任し、一定の精度を確保しました。
ところが運用開始から約 1 ヶ月後、予測値に異常が見られたため要因を調査したところ、

5 日前の日射量やその移動平均など、妥当性の低い説明変数が含まれていたことが判明しました。そこで説明変数を見直し、モデルを再構築しています。

バージョン	期間	説明変数	CV MAE
v1	～2026-05-17	8 変数 (GSR ・ TMP_max ・ 月 sin/cos ・ GSR_5day_avg ・ GSR_5day_ago 等)	—
v2	2026-05-18～06-06	6 変数 (GSR ・ GSR_ratio ・ TMP_max 等)	2.23 kWh
v3	2026-06-07～	3 変数 (GSR ・ GSR_ratio ・ SSD)	2.08 kWh
v3b	2026-06-23～	2 変数 (GSR ・ GSR_ratio、SSD 除 外)	2.22 kWh

変数を絞り込むたびに精度が向上しました (CV MAE 2.23→2.08 kWh)。

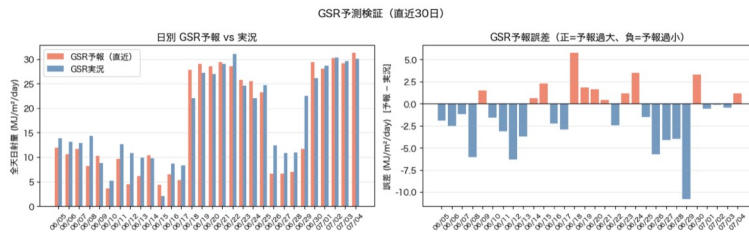
現行モデル (v3 / v3b 並行稼働)

2026-06-07 より v3 へ移行し、2026-06-23 より v3b を並行稼働させています。

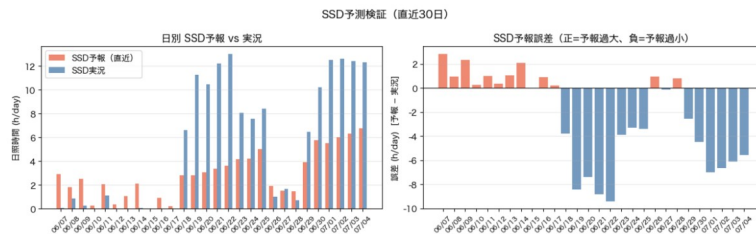
項目	v3 (SSD あり)	v3b (SSD なし)
説明変数	GSR ・ GSR/平年比 ・ SSD (3 変数)	GSR ・ GSR/平年比 (2 変数)
CV MAE	2.08 kWh/日	2.22 kWh/日
ファイル	rf_hatuden.joblib	rf_hatuden_no_ssd. joblib

並行稼働の理由は、SSD 予報が晴れ日に著しく過小評価 (翌日予報 bias=-7.7h/day) されており、v3 の発電量予測を系統的に引き下げている可能性を検証中のためです。

天気区分	v3 MAE（翌日）	v3b MAE（翌日）
晴れ日（SSD 実測 $\geq$ 5h）	3.68 kWh	1.73 kWh
曇り日（SSD 実測 $<$ 5h）	2.61 kWh	3.22 kWh



GSR（全天日射量）の予測と実測の検証



SSD（日照時間）の予測と実測の検証

発電量予測は蓄電池の充電判断にも活用しています（晴天時は太陽光優先、悪天候時は夜間電力）。個人宅では不要でも、大規模な太陽光＋蓄電池の運用では有効だと考えています。

5. バイブコーディングとは

「vibe coding」という言葉の由来

2025年2月、AI研究者のアンドレイ・カルパシー氏（元 Tesla AI 責任者・OpenAI 共同創業者）が X で提唱した言葉です。日本では通常そのまま「バイブコーディング」と呼ばれます。「vibe」は「雰囲気」や「ノリ」を意味し、「ノリで開発する」と説明されることもあります。

“There’s a new kind of coding I call ‘**vibe coding**’, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. [...] It’s not too bad for throwaway weekend projects.” — Andrej Karpathy (2025/2/2)

提唱者本人が「**使い捨ての週末プロジェクト向け**」と明言しており、当初は個人プロジェクトを念頭に置いたスタイルとして紹介されました。コードを一行ずつ書くのではなく、**見て・言って・動かして・貼り付ける**を繰り返し、アイデアを素早く形にするのが得意です。

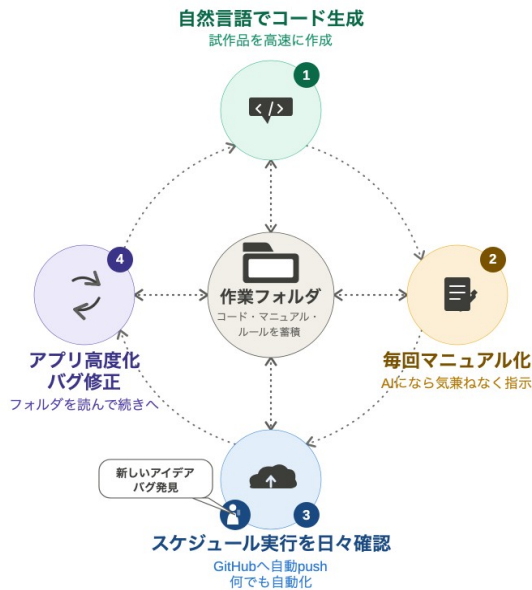
- **個人開発との相性:** 業務知識をもつ人が自らツールを試作しやすくなったことから、個人開発と相性が良いとされています。
- **企業での活用:** 試作品の作成や社内ツールの開発、アイデアの検証に活用されています。ただし利用者向けに使うには、人によるテストや改善が必要です。

vibe coding × GitHub — 私の開発スタイル

改めて自分の背景を整理すると、プログラミングを少しかじった程度で専門知識はなく、WXBC「アメダス気象データ分析チャレンジ」で Python・機械学習を学び、生成 AI によるプログラミングも同時期に始めて、この二つの学びが結びついた、というものです。

作業フォルダにコードとマニュアルを蓄積しながら開発するスタイルが定着しました。GitHub へ自動 push し、定時実行の結果をスマホから日々確認して、気づきを次の開発に活かします。フォルダに蓄積されたコードとマニュアルを AI が読むので、「続きから」で開発を再開できます。

このプレゼン資料自体もバイブコーディングで自動作成しました。ただし、学習が目的なら、AI に解説させながら開発を進める方が向いています。



私の開発サイクル。①自然言語でコード生成 → ②毎回マニュアル化 → ③スケジュール実行を日々確認・GitHub へ自動 push → ④アプリ高度化・バグ修正 → ①へ。中心には「作業フォルダ (コード・マニュアル・ルールを蓄積)」がある。

この開発サイクルのポイント

- **なぜ毎回プロンプトを入力し直さなくてよいのか:** コード・マニュアル・ルールを作業フォルダに置いておくと、次回の作業継続時に AI が自動で読み込みます。経緯を説明し直さず「続きから」で再開できます。
- **GitHub で閲覧するもの:** コードそのものというより、自動化処理した成果物 (Web アプリ・記事・グラフ・検証結果など) を見える化したものです。
- **バイブコーディングとしての強み:** 試作品の高速開発に向きます。コードの知識がなくても自然言語の指示だけで形にでき、「動くもの」を作りながら学べます。

自然言語でコード生成 → 毎回マニュアル化 → GitHub へ自動 push・定時実行 → アプリ高度化・バグ修正、が①に戻ってつながることで、開発が止まらずに続いていきます。

参考: 「[Claude Code ですべての日常業務を爆速化しよう!](#)」 (@minorun365, Qiita) — 3月に Claude Code を使い始めた際の最初の参考資料。

6. まとめ・今後の展開

- バイブコーディングは、開発の専門知識がなくても「作りたいもの」を言葉にできれば形にできる開発スタイルです。
- 気象データは公開されていて生活に密着しており、学習素材として最適です。WXBC の教材と AI を組み合わせれば、データの扱い方と AI 活用を同時に身につけられます。
- 開発経験がなくても、気象データへの興味と好奇心さえあれば形にできます。バイブコーディングは、WXBC が対象とする層に有効な開発手法だと考えています。
- 開発中の各ツールは、GitHub で成果を見える化し、運用しながら継続的に改善しています。
- 電力データ分析、太陽光発電予測の毎日実行、天気図生成、ローカル LLM、AI 最新情報や気象に関する情報の自動収集など、テーマを拡張中です。

以上